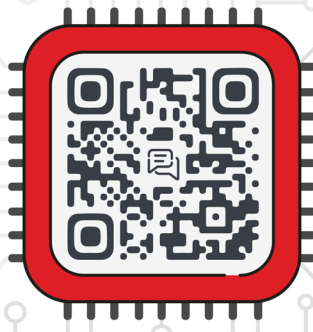




5



5



750 sp

نظري

كلية الهندسة المعلوماتية

السنة الأولى

Abstract classes & Interfaces

د. فراس ضعيف
د. معتز النجرس



البرمجة 2

RB Informatcs; 10/05/2026

مشكلة الأصناف العامة والحاجة إلى التجريد

في الأنظمة البرمجية، تظهر أحياناً حاجة إلى إنشاء أصناف أب ذات طابع عام جداً، مثل الصنف Shape الذي يمثل شكلاً هندسياً عاماً. تكمن المشكلة في أن بعض التوابع داخل هذا الصنف لا يمكن تحديد آلية عملها بدقة، لأن الصنف نفسه لا يمثل شكلاً محدداً.

■ مثال توضيحي: قصور الوراثة التقليدية

قصور الوراثة التقليدية هنا أن الصنف الابن قد يرث تابعاً عاماً من الصنف الأب دون أن يُجبر على إعادة تعريفه، رغم أن المنطق يتطلب تنفيذاً خاصاً به.

```
class Shape {
    public void draw() {
        System.out.println("Drawing... some shape?");
    }
}

class Circle extends Shape {
    private double radius;

    public Circle(double r) {
        this.radius = r;
    }

    @Override
    public void draw() {
        System.out.println("Drawing circle");
    }
}
```

■ تحليل سلوك الكود السابق

في حال إجراء Override، عند إنشاء كائن من الصنف الابن بالشكل الآتي:

```
Circle c = new Circle(5);
c.draw();
```

سيتم تنفيذ التابع الموجود داخل الصنف Circle، وبالتالي ستكون النتيجة:

Drawing circle

أما في حال نسيان إجراء Override داخل الصنف Circle، فلن يظهر خطأ من المترجم، لأن الصنف الابن سيرث التابع draw من الصنف الأب Shape. عندها سيؤدي استدعاء `c.draw()` إلى تنفيذ تابع الأب، وستكون النتيجة:

Drawing... some shape?

ويعد هذا السلوك صحيحاً من الناحية البرمجية، لكنه غير صحيح منطقياً في حالة الدائرة، لأن الدائرة يجب أن تمتلك طريقة رسم خاصة بها.

الأصناف والدوال المجردة Abstract Classes and Methods

لحل المشكلة السابقة، ولإجبار الأصناف الأبناء على إعادة تعريف بعض التوابع بما يناسب طبيعة كل صنف، يتم استخدام مفهوم التجريد Abstraction من خلال الكلمة المفتاحية `abstract`.

```
abstract class Shape {
    private String color;
    public Shape(String color) {
        this.color = color;
    }
    public abstract double getArea();
    public abstract double getPerimeter();
    public void describe() {
        System.out.println("I am a " + color + " shape.");
        System.out.printf("    Area      = %.2f\n", getArea());
        System.out.printf("    Perimeter = %.2f\n", getPerimeter());
    }
}

class Circle extends Shape {
    private double radius;
    public Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }
    @Override
    public double getArea() {
        return Math.PI * radius * radius;
    }
    @Override
    public double getPerimeter() {
        return 2 * Math.PI * radius;
    }
}
```



1. عند تعريف تابع مجرد بالشكل الآتي:

```
public abstract double getArea
```

فإن هذا يعني أن التابع لا يملك جسماً تنفيذياً داخل الصنف الأب، فالصنف الأب يحدد شكل التابع فقط دون تنفيذ، بينما تلزم الأصناف الأبناء بكتابة التنفيذ المناسب لكل تابع وفقاً لطبيعتها.

2. أي صنف غير مجرد يرث من صنف مجرد يجب أن يحقق جميع الدوال المجردة الموروثة منه. وفي حال لم يفعل ذلك، يصدر المترجم Compiler Error، إلا إذا تم إعلان الصنف الابن نفسه على أنه abstract.

3. تابع Override يجب أن يحمل الاسم نفسه، ونوع الإرجاع نفسه، وقائمة المعاملات نفسها الخاصة بالتابع المجرد. كما تحذف منه الكلمة abstract ويضاف له جسم تنفيذي بين الأقواس المجسمة.

4. لا يمكن إنشاء كائن مباشر من الصنف المجرد، لذلك لا يسمح بكتابة:

```
Shape s = new Shape("Red");
```

لأن Shape صنف مجرد.

5. يمكن أن يحتوي الصنف المجرد على باني Constructor. والغاية من هذا الباني هي تهيئة الخصائص المشتركة الموجودة في الصنف الأب، ويتم استدعاؤه من الصنف الابن باستخدام super، كما في مثال الخاصية color.

مقارنة منهجية: الوراثة التقليدية مقابل التجريد

يوضح الجدول الآتي الفرق الجوهرى بين الاعتماد على الوراثة التقليدية والاعتماد على التجريد باستخدام Abstract Class:

وجه المقارنة	الوراثة التقليدية Standard Inheritance	التجريد Abstraction via Abstract Class
الهدف الأساسي	إعادة استخدام الكود Code Reusability، إذ يرث الصنف الابن الدوال الجاهزة من الصنف الأب دون الحاجة إلى إعادة كتابتها.	فرض عقد برمجي Enforcing a Contract، إذ يجبر الصنف الأب الأصناف الأبناء على كتابة منطقها الخاص لبعض الدوال.
طبيعة الدوال	دوال ملموسة Concrete Methods تحتوي على كود تنفيذي افتراضي.	قد يحتوي الصنف المجرد على دوال مجردة بلا جسم تنفيذي، كما يمكن أن يحتوي أيضاً على دوال ملموسة مشتركة.
الإنشاء المباشر	يمكن إنشاء كائنات مباشرة من صنف الأب وصنف الابن، ما دام الصنف غير مجرد.	لا يمكن إنشاء كائن مباشر من الصنف الأب المجرد، ويكون إنشاء الكائنات من الأصناف الأبناء غير المجردة.
مثال عملي	الصنف Vehicle يحتوي على دالة startEngine جاهزة، فيرثها الصنف Car ويستخدمها مباشرة.	الصنف Vehicle يحتوي على دالة مجردة accelerate، فتضطر الأصناف Car و Truck إلى كتابة آلية التسارع الخاصة بكل منها.

الواجهات Interfaces وحل معضلة الوراثة المتعددة

المشكلة: محدودية الوراثة في Java

تمنع لغة Java الوراثة المتعددة من الأصناف، أي إن الصنف الواحد لا يستطيع أن يرث من أكثر من صنف باستخدام extends. ويهدف هذا القيد إلى تجنب التعارضات التي قد تنتج عن وراثة أكثر من تنفيذ للدالة نفسها من أكثر من صنف أب.

لنفترض وجود ثلاث دوال مستقلة وعامة، مع الحاجة إلى بناء ثلاثة أصناف بحيث يحتاج كل صنف إلى دالتين فقط بتوزيع مختلف. على سبيل المثال، الصنف الأول يحتاج إلى الدالتين 1 و2، والصنف الثاني يحتاج إلى الدالتين 2 و3، والصنف الثالث يحتاج إلى الدالتين 1 و3. في هذه الحالة لن تكون الوراثة التقليدية مناسبة، لأن كل صنف في Java لا يمكنه أن يمتد إلا من صنف واحد فقط باستخدام extends.

الحل البرمجي: الواجهة Interface

الواجهة ليست صنفاً تقليدياً، بل تمثل مخططاً أو عقداً برمجياً يحدد ما يجب على الصنف المنفذ أن يحققه. لا تحتوي الواجهة على بواني Constructors، ولا يمكن إنشاء كائنات منها مباشرة.

في صورتها الأساسية، تكون الدوال المعلنة داخل الواجهة دون جسم تنفيذي دوالاً عامة ومجردة تلقائياً implicitly public abstract. كما تسمح Java للصنف الواحد بأن ينفذ أكثر من واجهة في الوقت نفسه باستخدام implements.

```
interface Printable {
    void print();
}
interface Savable {
    void save(String filename);
    void load(String filename);
}
class Report implements Printable, Savable {
    private String title;
    private String content;
    public Report(String title, String content) {
        this.title = title;
        this.content = content;
    }
    @Override
    public void print() {
        System.out.println("=== " + title + " ===");
        System.out.println(content);
    }
    @Override
    public void save(String filename) {
        System.out.println("Saving '" + title + "' to " + filename);
    }
}
```

```
@Override
public void load(String filename) {
    System.out.println("Loading from " + filename);
}
}
```

في المثال السابق، قام الصنف Report بتنفيذ الواجهتين Printable و Savable معاً. وبذلك أصبح ملزماً بتحقيق جميع الدوال المعلنة داخلهما، وهي:

```
print()
save(String filename)
load(String filename)
```

وهذا يوضح دور الواجهات في تمكين الصنف الواحد من الالتزام بأكثر من عقد برمجي، دون الحاجة إلى وراثة متعددة من الأصناف.

